

AVL TREE – FULL NOTES

1. Definition

An **AVL Tree** is a **self-balancing Binary Search Tree** where the **height difference** between the left and right subtrees of **any node** is at most **1**.

Discovered by **Adelson-Velsky and Landis** → hence the name **AVL**.

2. Why AVL Tree? (Need)

A normal BST can become **skewed** (like a linked list) → worst-case $O(n)$.

AVL maintains balance → **guaranteed $O(\log n)$** time for:

- ✓ Searching
- ✓ Insertion
- ✓ Deletion

3. Balance Factor (MOST IMPORTANT)

For any node:

Balance Factor = height(left subtree) - height(right subtree)

A node is **balanced** if:

$BF \in \{ -1, 0, +1 \}$

If BF is **+2 or -2**, the tree is **unbalanced**, and rotation is needed.

4. Properties of AVL Tree

1. BST property holds
Left < Root < Right
2. Balance Factor of every node is -1, 0, or +1.
3. Height is always $\log(n)$.
4. Insertion and deletion use rotations to maintain balance.

5. Time Complexities

Operation	Time Complexity
-----------	-----------------

Search	$O(\log n)$
--------	-------------

Insert	$O(\log n)$
--------	-------------

Delete	$O(\log n)$
--------	-------------

Space	$O(n)$
-------	--------

6. Rotations in AVL Tree (VERY IMPORTANT)

To fix the imbalance, AVL performs **four types of rotations**:

Case 1: LL Rotation (Left Left)

Occurs when:

- Node inserted in **left subtree of left child**

Fix → **Right Rotation**



Case 2: RR Rotation (Right Right)

Occurs when:

- Node inserted in **right subtree of right child**

Fix → **Left Rotation**



Case 3: LR Rotation (Left Right)

Occurs when:

- Node inserted in **right subtree of left child**

Fix →

1. Left Rotate on child
2. Right Rotate on parent

z
/
y
\
x

Case 4: RL Rotation (Right Left)

Occurs when:

- Node inserted in **left subtree of right child**

Fix →

1. Right Rotate on child
2. Left Rotate on parent

z
\
y
/
x

7. AVL Insertion (Step-by-Step)

1. Insert node like a normal BST
2. Update height
3. Compute the balance factor
4. If **imbalanced**, apply rotation

Insertion Imbalance Cases

Case	Condition	Rotation
LL	$BF > 1 \ \&\& \ key < left.key$	Right Rotate
RR	$BF < -1 \ \&\& \ key > right.key$	Left Rotate
LR	$BF > 1 \ \&\& \ key > left.key$	Left + Right Rotate
RL	$BF < -1 \ \&\& \ key < right.key$	Right + Left Rotate

8. AVL Deletion

Steps:

1. Delete like BST
2. Update heights
3. Check balance factor
4. Apply appropriate rotation

Deletion may cause **multiple imbalances** on the way up.

9. Example of LL Case

Insert numbers: 30, 20, 10

30
/
20
/
10

This is LL → perform **Right Rotate** on 30:

20
/ \
10 30

10. Example of RR Case

Insert: 10, 20, 30

10
\
20
\
30

RR → Left Rotate → becomes:



11. Advantages of AVL Tree

- ✓ Always balanced
- ✓ Better search performance than Red-Black Tree
- ✓ Guaranteed $O(\log n)$ height
- ✓ Useful for dictionary lookups

12. Disadvantages

- ✗ More rotations than Red-Black Tree
- ✗ Insertion/Deletion time slightly higher
- ✗ Implementation is complex

13. Applications

- Database indexing
- Memory allocation
- File systems
- Maintaining sorted data
- Real-time systems requiring fast lookup

16. Exam/Interview Important Questions

1. What is an AVL Tree? Why is it needed?
2. Define balance factor. What values are allowed?
3. Explain the four rotations in the AVL Tree.

4. When do we perform LL, RR, LR, and RL rotations?
5. Insert the following keys and construct an AVL Tree:
 - Example: 30, 10, 20
6. Explain AVL deletion with an example.
7. Compare AVL and Red-Black Trees.
8. What is the time complexity of an AVL Tree?
9. Why does AVL provide better search than Red-Black?
10. What happens when the balance factor becomes +2 or -2?